



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе №1
по курсу «Анализ Алгоритмов»
на тему: «Расстояние Левенштейна»

Студент ИУ7-55Б
(Группа)

(Подпись, дата)

Казарян Д. Д.
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Волкова Л. Л.
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Расстояние Левенштейна	4
1.2 Нерекурсивный алгоритм нахождения расстояния Левенштейна	5
1.3 Рекурсивный алгоритм нахождения расстояния Левенштейна .	5
1.4 Рекурсивный алгоритм нахождения расстояния Левенштейна с кэшем	5
1.5 Расстояние Дameraу — Левенштейна	5
2 Конструкторская часть	7
2.1 Разработка алгоритмов	7
3 Технологическая часть	12
3.1 Средства реализации	12
3.2 Сведения о модулях программы	12
3.3 Реализация алгоритмов	12
3.4 Функциональные тесты	13
4 Исследовательская часть	14
4.1 Технические характеристики	14
4.2 Время выполнения реализаций алгоритмов	14
ЗАКЛЮЧЕНИЕ	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18
ПРИЛОЖЕНИЕ А	19

ВВЕДЕНИЕ

Целью данной лабораторной работы является исследование алгоритмов поиска расстояний Левенштейна и Дamerau — Левенштейна.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- 1) описать алгоритмы поиска расстояний Левенштейна и Дamerau — Левенштейна для нахождения редакционного расстояния между строками;
- 2) реализовать данные алгоритмы;
- 3) выполнить сравнительный анализ алгоритмов по затрачиваемым ресурсам (времени, памяти);
- 4) описать и обосновать полученные результаты в отчете.

1 Аналитическая часть

В данном разделе будут представлены описания алгоритмов нахождения расстояний Левенштейна и Дамерау — Левенштейна и их практическое применение.

1.1 Расстояние Левенштейна

Расстояние Левенштейна — это метрика между двумя строками. Оно равно минимальному числу элементарных операций (вставок, удалений и замен символов), необходимых для преобразования одной строки в другую[1]. Каждая операция имеет свою цену(w). Редакционное предписание — это последовательность операций с суммарной минимальной стоимостью, которую необходимо выполнить для получения из первой строки вторую. Эта цена и есть искомое расстояние Левенштейна.

Используются следующие обозначения:

- 1) I (от англ. insert) — вставка ($w(\lambda, b) = 1$);
- 2) R (от англ. replace) — замена ($w(a, b) = 1, a \neq b$);
- 3) D (от англ. delete) — удаление ($w(a, \lambda) = 1$).

Функция $D(i, j)$ определяет редакционное расстояние между строками $S_1[1...i]$ и $S_2[1...j]$.

Расстояние Левенштейна между двумя строками S_1 и S_2 (длиной M и N соответственно) рассчитывается по следующей рекуррентной формуле:

$$D(i, j) = \begin{cases} 0, & i = 0, j = 0 \\ i, & j = 0, i > 0 \\ j, & i = 0, j > 0 \\ \min \begin{cases} D(i, j - 1) + 1, \\ D(i - 1, j) + 1, \\ D(i - 1, j - 1) + m(S_1[i], S_2[j]), \end{cases} & i > 0, j > 0 \end{cases} \quad (1.1)$$

где сравнение символов строк S_1 и S_2 рассчитывается таким образом:

$$m(a, b) = \begin{cases} 0 & \text{если } a = b, \\ 1 & \text{иначе.} \end{cases} \quad (1.2)$$

1.2 Нерекурсивный алгоритм нахождения расстояния Левенштейна

Рекурсивная реализация алгоритма нахождения расстояния Левенштейна малоэффективна по времени при больших $N, M > 10$, так как производится много повторных вычислений. Для оптимизации применяется матрица размерности $(N + 1) \times (M + 1)$, в которой будут храниться промежуточные значения. Первый элемент матрицы заполнен нулём. Остальные требуется заполнить в соответствии с формулой (1.1). Результатом работы будет являться значение в последней ячейке последней строки матрицы. Для вычислений необязательно хранить всю матрицу — достаточно двух строк.

1.3 Рекурсивный алгоритм нахождения расстояния Левенштейна

Рекурсивный алгоритм реализует формулу (1.1).

1.4 Рекурсивный алгоритм нахождения расстояния Левенштейна с кэшем

Рекурсивный алгоритм заполнения оптимизируется по времени выполнения с использованием кеша. В качестве кеша используется матрица. В случае, если рекурсивный алгоритм выполняет расчёт для данных, которые ещё не были обработаны (в ячейке находится бесконечность), результат нахождения расстояния заносится в матрицу. В случае, если ячейка не равна бесконечности, расстояние не находится и алгоритм переходит к следующему шагу.

1.5 Расстояние Дамерау — Левенштейна

Расстояние Дамерау — Левенштейна — это метрика между двумя строками. Оно равно минимальному числу элементарных операций (вставок, удалений, замен и перестановок символов), необходимых для преобразования одной

строки в другую [1]. Является расширением расстояния Левенштейна, поскольку помимо трёх базовых операций содержит ещё операцию транспозиции T (от англ. transposition).

Расстояние Дамерау — Левенштейна определяется следующей рекуррентной формуле:

$$D(m, n) = \begin{cases} 0, & i = 0, j = 0 \\ i, & j = 0, i > 0 \\ j, & i = 0, j > 0 \\ \min \begin{cases} D(i, j - 1) + 1, \\ D(i - 1, j) + 1, \\ D(i - 1, j - 1), \\ D(i - 2, j - 2) + 1, \end{cases} & \begin{aligned} & \text{если } i, j > 1, \\ & S_1[i] = S_2[j - 1], \\ & S_1[i - 1] = S_2[j], \end{aligned} \\ \min \begin{cases} D(i - 1, j) + 1, \\ D(i, j - 1) + 1, \\ D(i - 1, j - 1) + m(S_1[i], S_2[j]) \end{cases} & \text{иначе.} \end{cases} \quad (1.3)$$

Вывод

В данной части были рассмотрены формулы Левенштейна и Дамерау — Левенштейна, используемые для вычисления расстояния между строками. Обе формулы задаются рекуррентно, что позволяет реализовать как рекурсивный, так и итеративный подходы.

2 Конструкторская часть

В этом разделе будут представлены схемы алгоритмов для вычисления расстояний Левенштейна и Дамерау — Левенштейна.

2.1 Разработка алгоритмов

На рисунке 2.1 приведена схема рекурсивного алгоритма нахождения расстояния Левенштейна.

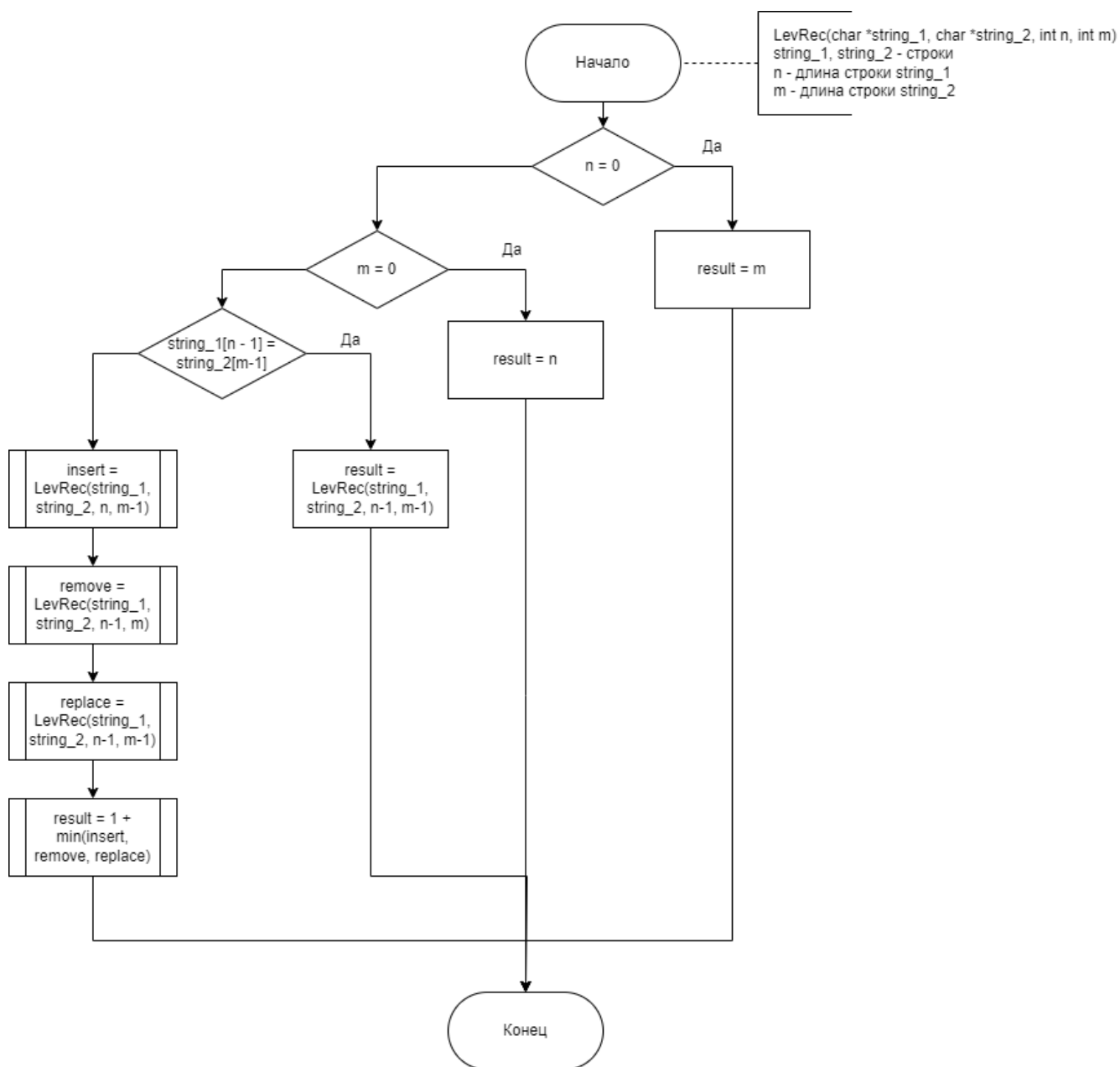


Рисунок 2.1 – Схема рекурсивного алгоритма нахождения расстояния Левенштейна.

На рисунке 2.2 приведена схема рекурсивного алгоритма нахождения расстояния Левенштейна с кэшем.

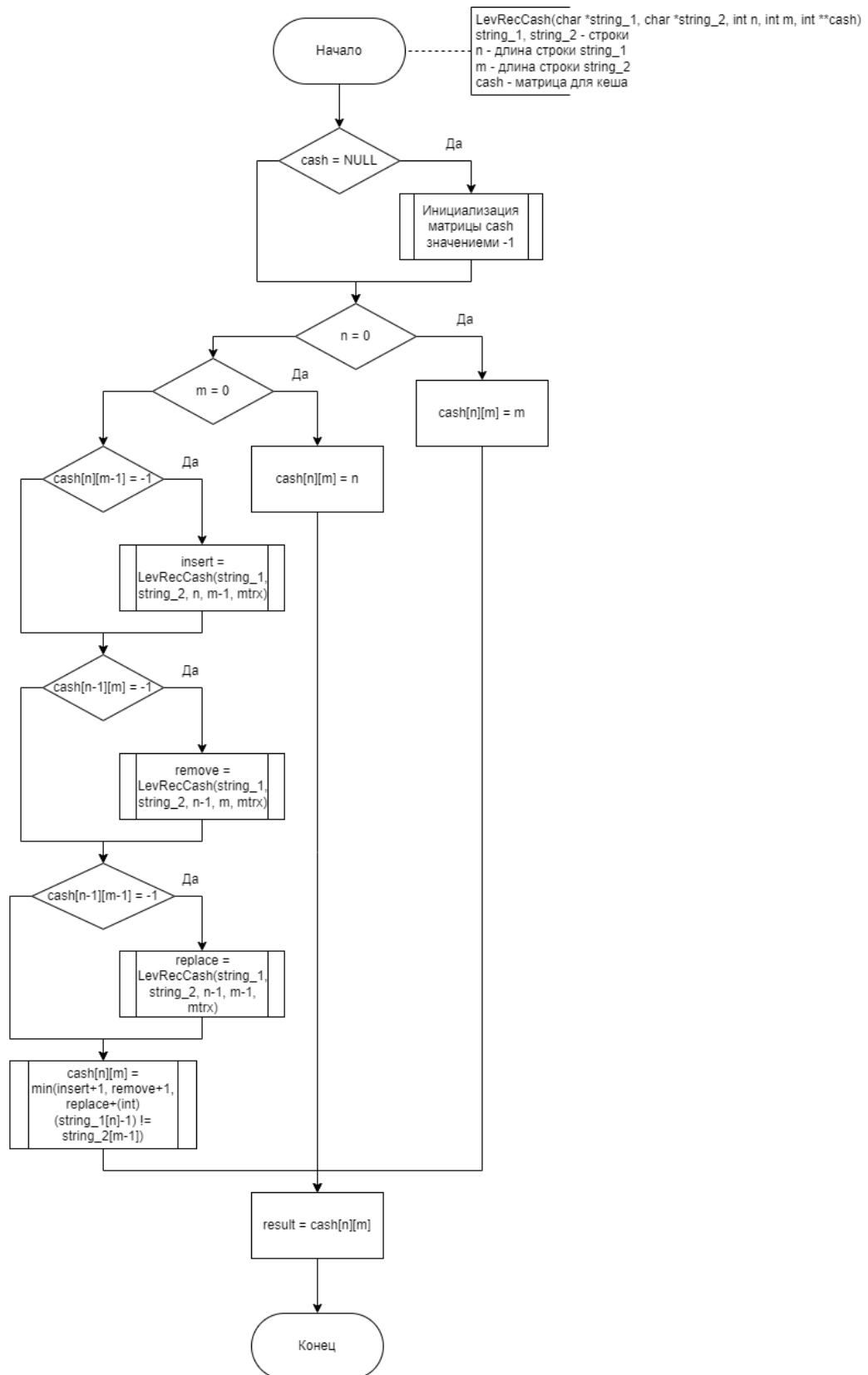


Рисунок 2.2 – Схема не рекурсивного алгоритма нахождения расстояния Левенштейна с кэшем.

На рисунке 2.3 приведена схема нерекурсивного алгоритма нахождения расстояния Левенштейна.

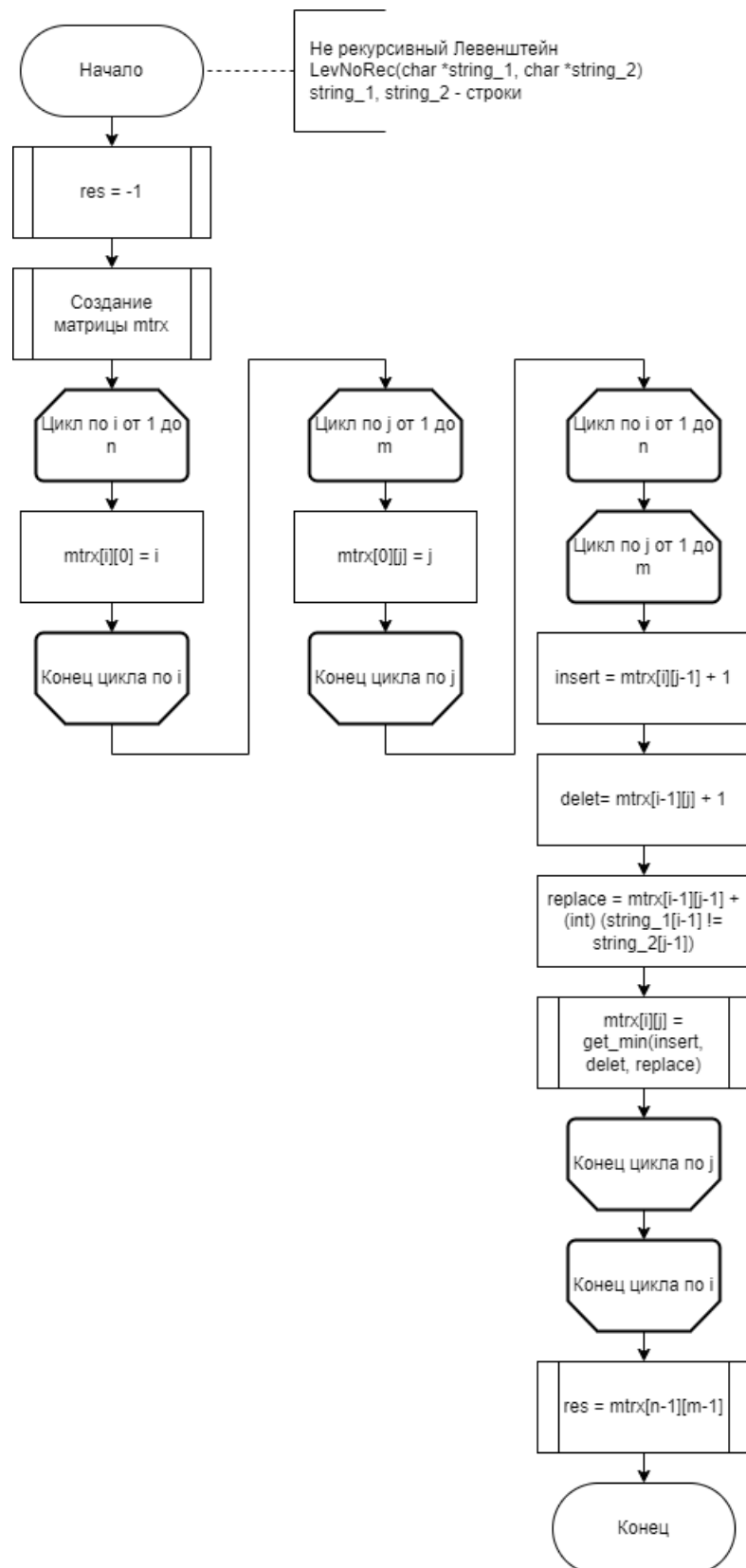


Рисунок 2.3 – Схема нерекурсивного алгоритма нахождения расстояния Левенштейна.

На рисунке 2.4 приведена схема нерекурсивного алгоритма нахождения расстояния Дамерау — Левенштейна.

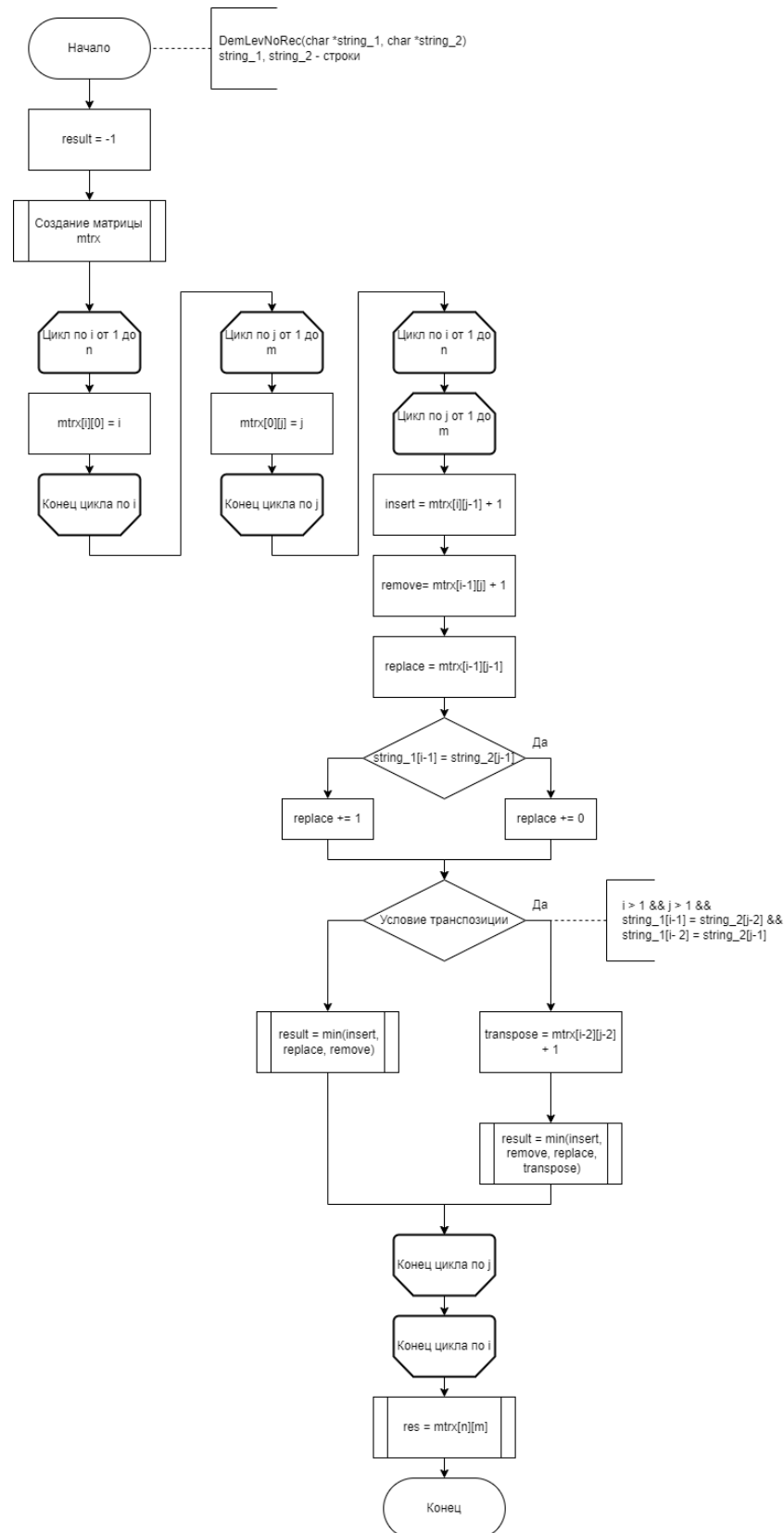


Рисунок 2.4 – Схема нерекурсивного алгоритма нахождения расстояния Дамерау — Левенштейна

Вывод

В данном разделе были разработаны схемы необходимых алгоритмов, опираясь на данные, представленные в аналитической части.

3 Технологическая часть

В данном разделе будут приведены средства реализации, листинг кода и функциональные тесты.

3.1 Средства реализации

В качестве языка программирования для реализации данной лабораторной работы был выбран язык программирования C [2]

В этой лабораторной работе необходимо измерить время выполнения процессора для каждой реализации алгоритма и построить графики, показывающие зависимость времени выполнения от длины строк. Все необходимые инструменты для выполнения этой задачи доступны в выбранном языке программирования.

Время работы замерено с помощью функции `clock()` из библиотеки `time` [3].

3.2 Сведения о модулях программы

Данная программа разбита на следующие модули:

- 1) `main.c` — главный файл программы содержащий точку входа в программу, где располагается вывод меню.
- 2) `alloc_matrix.c` — файл, содержащий реализацию динамической матрицы.
- 3) `algorithms.c` — файл с реализацией исследуемых алгоритмов.
- 4) `measurement.c` — файл с функциями, замеряющими время выполнения алгоритмов поиска расстояния Левенштейна и Дамерау — Левенштейна.

3.3 Реализация алгоритмов

В листингах A.1 – A.4 в приложении A приведены реализации алгоритмов поиска расстояний Левенштейна и Дамерау — Левенштейна.

3.4 Функциональные тесты

В таблице 3.1 представлены функциональные тесты для реализованных алгоритмов. Тестирование проведено по методу чёрного ящика. Все тесты пройдены успешно.

Таблица 3.1 – Функциональные тесты

Входные данные			Ожидаемый результат	
№	Строка 1	Строка 2	Левенштейн	Дамерау — Левенштейн
1	ab	ab	0	0
2	aa	df	2	2
3	asdfg	sof	3	3
4	string	slakng	3	3
5	dog	dooog	2	2
6	dog	doggy	2	2
7	dog	god	2	2
8	"пустая строка"	"пустая строка"	0	0
9	lack	nolack	2	2
10	kronen	lampad	6	6

Вывод

В данном были разработаны и протестированы реализации алгоритмов нахождения расстояния Левенштейна и Дамерау — Левенштейна.

4 Исследовательская часть

В данном разделе будут приведены примеры работы программ и сравнительный анализ алгоритмов на основе полученных данных.

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось исследование:

- операционная система: Windows 11 Home Single Language 23H2;
- память: 16 Гб;
- процессор: 1,8 ГГц 4-ядерный процессор Intel Core i7.

Во время тестирования устройство было подключено к сети электропитания, нагружено приложениями окружения и самой системой тестирования.

4.2 Время выполнения реализаций алгоритмов

Замеры процессорного времени выполнения реализаций алгоритмов произведены при помощи функции `clock()` из библиотеки `time` языка C. Данная функция возвращает количество временных тактов, прошедших с начала запуска программы.

Замеры проводились для строк одинаковой длины от 0 до 12 символов по 1000 раз (Рисунок 4.1) до 1000 символов (Рисунок 4.2).

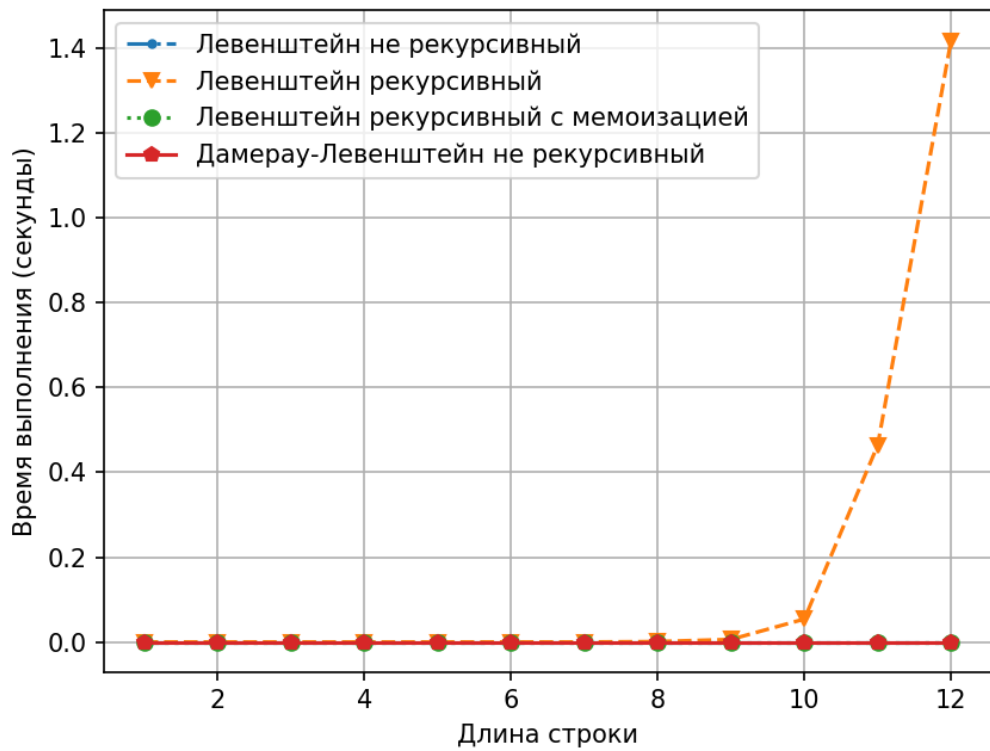


Рисунок 4.1 – Сравнение по времени выполнения реализаций алгоритмов нахождения расстояния Левенштейна

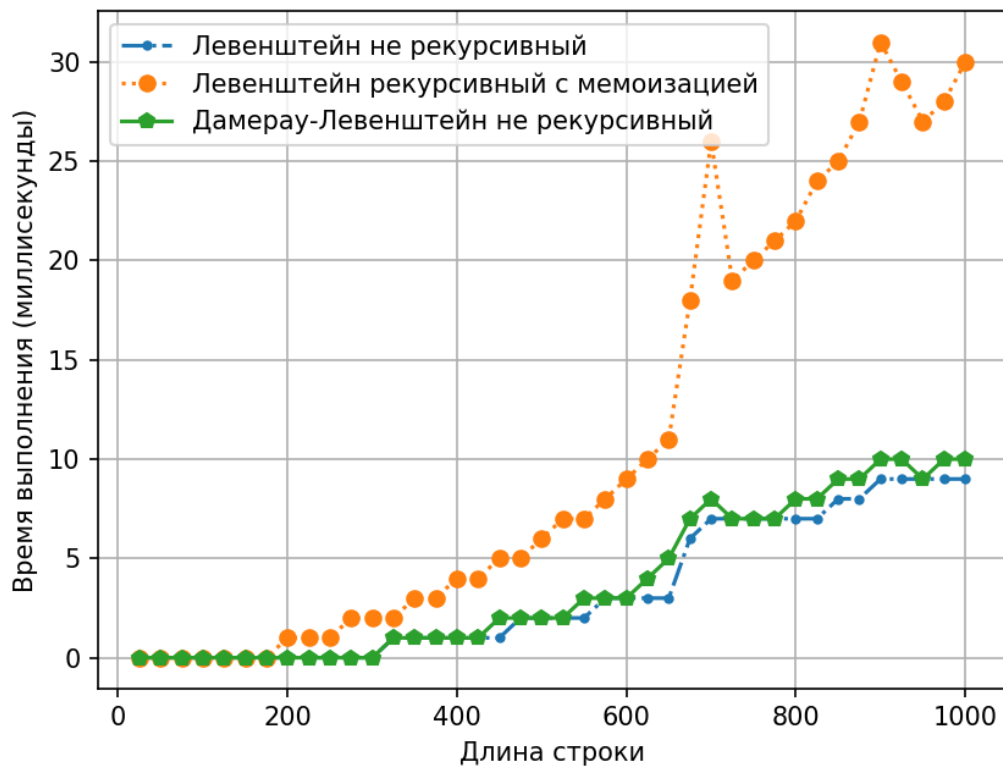


Рисунок 4.2 – Сравнение по времени выполнения нерекурсивной и рекурсивной с кэшем реализаций алгоритмов нахождения расстояния Левенштейна с не рекурсивной реализацией алгоритма Дамерау — Левенштейна

Вывод

Исходя из замеров по времени, рекурсивная реализация алгоритма нахождения расстояния Левенштейна работает дольше остальных. Наименее затратным по времени оказалась не рекурсивная реализация алгоритма нахождения расстояния Левенштейна.

Исходя из замеров по памяти, не рекурсивные и рекурсивные с кэшем реализации алгоритмов проигрывают рекурсивным.

ЗАКЛЮЧЕНИЕ

Исследование показало, что время выполнения алгоритмов Левенштейна и Дамерау — Левенштейна растёт геометрически с увеличением длины строк. Нерекурсивная реализация и рекурсивная с кэшем для алгоритма Левенштейна показывают лучшее время.

Цель лабораторной работы была достигнута: исследованы алгоритмы поиска расстояний Левенштейна и Дамерау — Левенштейна.

Выполнены следующие задачи:

- 1) описать алгоритмы поиска расстояний Левенштейна и Дамерау — Левенштейна для нахождения редакционного расстояния между строками;
- 2) реализовать данные алгоритмы;
- 3) выполнить сравнительный анализ алгоритмов по затрачиваемым ресурсам (времени);
- 4) описать и обосновать полученные результаты в отчете.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Таразанов А. М. П. Д. А. и.* Сравнительный анализ алгоритмов редакционного расстояния Левенштейна и Дамерау — Левенштейна // Синергия Наук. — 2019.
2. C C++ reference [Электронный ресурс]. — Режим доступа: <https://en.cppreference.com/w/> (дата обращения: 07.01.2025).
3. time — Time access and conversions [Электронный ресурс]. — Режим доступа: <https://en.cppreference.com/w/c/chrono> (дата обращения: 07.01.2025).

ПРИЛОЖЕНИЕ А

Листинг А.1 – Реализация рекурсивного алгоритма поиска расстояния Левенштейна

```
1 int LevRec(char *string_1, char *string_2, int n, int m)
2 {
3     if (n == 0) return m;
4     if (m == 0) return n;
5
6     if (string_1[n - 1] == string_2[m - 1])
7         return LevRec(string_1, string_2, n - 1, m - 1);
8
9     int insert = LevRec(string_1, string_2, n, m - 1);
10    int remove = LevRec(string_1, string_2, n - 1, m);
11    int replace = LevRec(string_1, string_2, n - 1, m - 1);
12
13    return 1 + get_min(insert, remove, replace);
14 }
```

Листинг А.2 – Реализация рекурсивного алгоритма поиска расстояния Левенштейна с кэшем

```
1  int LevRecCashIter(char *string_1, char *string_2, int n, int m,
   int **cash)
2  {
3      if (cash == NULL) return -1;
4      if (n == 0) return m;
5      else if (m == 0) return n;
6      else if (cash[n][m] != -1) return cash[n][m];
7      int cost = (string_1[n-1] != string_2[m-1]) ? 1 : 0;
8      int insert = LevRecCashIter(string_1, string_2, n, m - 1,
   cash) + 1;
9      int remove = LevRecCashIter(string_1, string_2, n - 1, m,
   cash) + 1;
10     int replace = LevRecCashIter(string_1, string_2, n - 1, m -
   1, cash) + cost;
11     cash[n][m] = get_min(
12         insert,
13         remove,
14         replace
15     );
16
17     return cash[n][m];
18 }
```

Листинг А.3 – Реализация нерекурсивного алгоритма поиска расстояния Левенштейна

```
1 int LevNoRec(char *string_1, char *string_2)
2 {
3     int res = -1;
4
5     int n = strlen(string_1) + 1;
6     int m = strlen(string_2) + 1;
7
8     int **mtrx = create_mtrx(n, m);
9
10    if (mtrx != NULL)
11    {
12        mtrx[0][0] = 0;
13        for (int i = 1; i < n; i++)
14            mtrx[i][0] = i;
15        for (int j = 1; j < m; j++)
16            mtrx[0][j] = j;
17
18        int insert = 0;
19        int remove = 0;
20        int replace = 0;
21
22        for (int i = 1; i < n; i++)
23            for (int j = 1; j < m; j++)
24            {
25                insert = mtrx[i][j - 1] + 1;
26                remove = mtrx[i - 1][j] + 1;
27                replace = mtrx[i - 1][j - 1] + (int)(string_1[i
28                    - 1] != string_2[j - 1]);
29
30                mtrx[i][j] = get_min(insert, remove, replace);
31            }
32
33        res = mtrx[n - 1][m - 1];
34        delete_mtrx(mtrx, n);
35    }
36
37    return res;
38 }
```

Листинг А.4 – Реализация нерекурсивного алгоритма поиска расстояния
Дамерау — Левенштейна

```
1  int DemLevNoRec(char *string_1, char *string_2)
2  {
3      int result = -1;
4      int n = strlen(string_1) + 1;
5      int m = strlen(string_2) + 1;
6      int **mtrx = create_mtrx(n, m);
7      if (mtrx != NULL)
8      {
9          mtrx[0][0] = 0;
10         for (int i = 1; i < n; i++)
11             mtrx[i][0] = i;
12         for (int j = 1; j < m; j++)
13             mtrx[0][j] = j;
14
15         for (int i = 1; i < n; i++)
16             for (int j = 1; j < m; j++)
17             {
18                 int cost = string_1[i - 1] == string_2[j - 1] ?
19                     0 : 1;
20                 mtrx[i][j] = get_min(mtrx[i - 1][j] + 1,\
21                     mtrx[i][j - 1] + 1,\
22                     mtrx[i - 1][j - 1] + cost);
23                 if (i > 1 && j > 1 && string_1[i - 1] ==
24                     string_2[j - 2] && string_1[i - 2] ==
25                     string_2[j - 1])
26                 {
27                     int buff = mtrx[i - 2][j - 2] + cost;
28                     if (buff < mtrx[i][j])
29                         mtrx[i][j] = buff;
30                 }
31             }
32         result = mtrx[n - 1][m - 1];
33         delete_mtrx(mtrx, n);
34     }
35     return result;
36 }
```